Session 3 · Debugging, Testing & Financial Analytics

The difference between code that runs and code you can trust. You repair a broken version of Session 2's Apple valuation with Claude's help, protect it with sanity checks, and finish with an investment-memo draft produced by your own engine — every claim machine-verified.

Plan: concepts 12' · live demo 22' · your lab 30' · debrief 8'

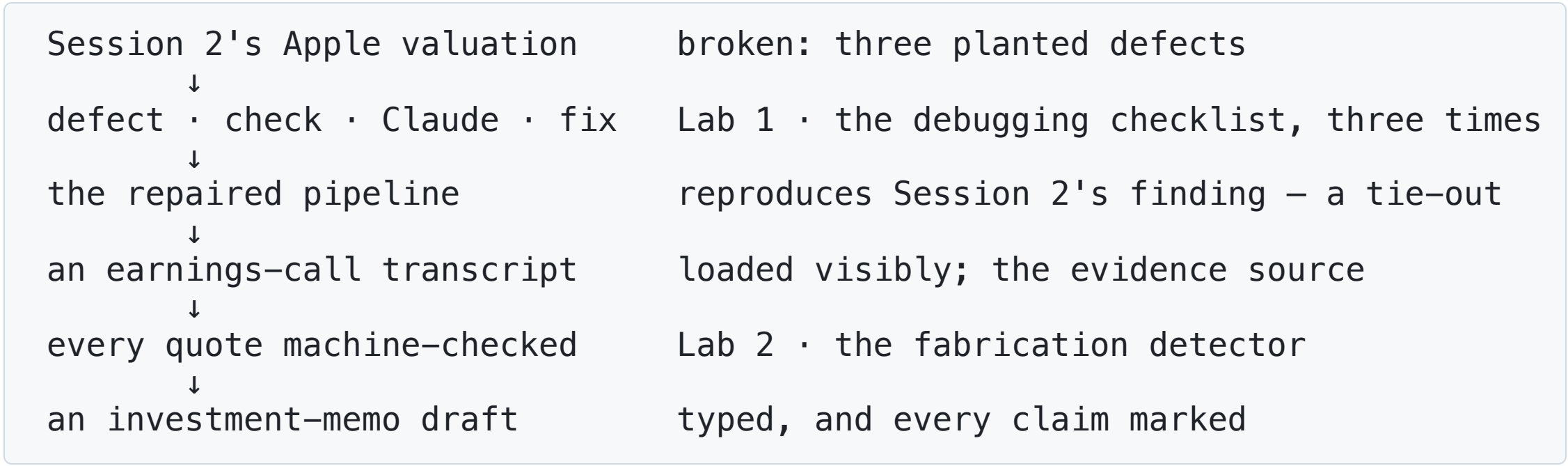
By the end of Session 3 you can

1. **Diagnose** an error with Claude: reproduce, explain the cause, fix, verify
2. **Write** sanity checks for financial calculations: magnitudes, units, tie-outs
3. **Build** an engine that turns a whole earnings call into structured analysis
4. **Verify** every quoted claim automatically, and produce a memo draft

What we are doing, and why

- **Code that runs is not code you can trust.** All three of today's planted defects run without any error — and produce absurd numbers.
- **Trust comes from checks, not from confidence**: order-of-magnitude limits, unit consistency, and **tie-outs** against publicly known figures. Apple's market value is about $4.5 trillion; a result of $4.5 billion is enough to reject the code.
- **The debugging discipline is the same with AI as without**: reproduce, isolate, **make Claude explain the cause before fixing**, change one line, verify with a check.
- **Then the same discipline for AI-written analysis**: an engine reads an earnings call, and three gates guard its output — sanity checks catch wrong *numbers*, Pydantic catches wrong *shapes*, and your quote-checker catches wrong *evidence*. Together they are **the trust layer**.

The storyline of this session



The four sanity checks of financial code

Check	Question it asks	Today's example
Order of magnitude	can this number exist?	a 119% operating margin cannot
Unit consistency	millions, billions, or units?	market value off by a factor of 1,000
Tie-out	does it match a known figure?	Apple ≈ $4.5 trillion, publicly known
Loud failure	did bad data stop the pipeline?	a merge that silently produces missing values

Every check is one `assert` with a message that names the fix.

The demonstration: three defects, one checklist

Defect	Symptom	The check that catches it
Wrong formula	margin divided by the wrong base	order of magnitude
Silent data problem	one dirty ticker, missing values after a merge	loud failure at the boundary
Unit error	share count "converted" wrongly	tie-out against the known figure

For each: run it, read the implausible output aloud, ask Claude to **explain the cause before proposing a fix**, repair one line, and let the check decide.

The earnings engine: find the fabricated quote

```
transcript → model extracts claims → every claim carries a verbatim quote
           → your code checks each quote against the document
           → Pydantic validates the shape (typed fields, precise rejections)
           → a memo, every claim marked verified or not
```

- **The game**: the analysis you will see contains eleven quoted claims, and exactly one quote is fabricated — fluent, plausible, in the transcript's own style. Reading rarely finds it; ten lines of Python do.
- **The company is fictional by design**: only a fictional call proves the claims come from the *document*, not the model's memory — the engine runs unchanged on any real transcript

The libraries in this session

Library	Role here	Standing
`pandas`	the pipeline under repair	the industry standard for data work
`pydantic`	the shape gate: typed models, addressed rejections	the industry standard for validation
`anthropic`	schema-forced calls to Claude	the official Claude SDK

How the labs work

Every exercise sits between two markers. **You fill the gaps. Nothing else changes.**

```
### START CODE HERE ###
df["op_margin"] = df["operating_income_m"] / df[None]   # a margin divides profit by what?
### END CODE HERE ###
```

- `None` → replace with the correct column, value or variable
- `[QUESTION IN CAPITALS]` → replace with the text the bracket asks for
- **Everything else is given.** Do not rewrite it.
- Then run the ✅ **check cell** directly below. Green means correct: continue.

Stuck for two minutes? Select the lines, press `Option+K` (`Alt+K` on Windows), and ask the ✱ panel.

Your lab · notebook 03 · 30 minutes

- **Lab 1**: three defects in Session 2's Apple valuation — for each, read the symptom, make Claude explain the cause, fix one line, pass the check
- **The tie-out**: the repaired pipeline reproduces Session 2's finding
- **Lab 2**: the analysis holds eleven quoted claims; one quote is a lie — build the detector that finds it, then generate your memo draft
- **Milestone**: the engine flags **exactly one** planted fabricated quote

Key takeaway

Professionals are distinguished not by writing perfect code, but by knowing how to prove their numbers are right.

Next session: retrieving filings programmatically from the U.S. Securities and Exchange Commission.